

CSE207

Linked List Quiz Practice Problem Set

Singly Linked List and Doubly Linked List | Summer 2026

Course	CSE207	Language	C or C++
--------	--------	----------	----------

Instructions: Solve using C or C++. Define suitable node structures, handle empty lists and boundary cases, update every affected pointer, and free deleted nodes. Do not use arrays or library list containers unless explicitly allowed.

Part A: Singly Linked List**1. Create and Display a List**

Problem: Create a singly linked list from n integers and display its elements in order.

Requirement: Use separate functions for creating a node, inserting at the end, and displaying the list. Print Empty list when n = 0.

Example: Input: 4 values 10 20 30 40 -> 10 20 30 40

2. Count Nodes and Find the Sum

Problem: Traverse a singly linked list and print the number of nodes and the sum of all stored values.

Requirement: Use one traversal and do not store the values in an array.

Example: List: 4 7 9 -> Count = 3, Sum = 20

3. Insert at a Given Position

Problem: Insert a new node at a user-specified 1-based position.

Requirement: Support insertion at the beginning, middle, and end; reject positions outside 1 to length + 1.

Example: List: 10 20 40; insert 30 at position 3 -> 10 20 30 40

4. Delete Every Occurrence of a Key

Problem: Delete all nodes containing a given key.

Requirement: Handle matches at the head, consecutive matches, a missing key, and a list that becomes empty. Free removed nodes.

Example: List: 5 8 5 5 12 5; key = 5 -> 8 12

5. Merge Two Sorted Singly Linked Lists

Problem: Merge two ascending singly linked lists into one ascending list.

Requirement: Reuse the existing nodes and relink pointers; do not create a third set of data nodes or copy values into an array.

Example: A: 1 4 7; B: 2 3 8 9 -> 1 2 3 4 7 8 9

6. Reverse the List In-Place

Problem: Reverse a singly linked list without changing the data stored in its nodes.

Requirement: Use an iterative pointer-based algorithm with $O(1)$ extra space.

Example: 1 2 3 4 5 $\rightarrow$ 5 4 3 2 1

7. Find the Middle and K-th Node from the End

Problem: Print the middle node and the K-th node from the end of a singly linked list.

Requirement: Use slow/fast pointers for the middle and a two-pointer gap for K-th from the end. Do not count the list first.

Example: List: 10 20 30 40 50; $K = 2 \rightarrow$ Middle = 30, K-th from end = 40

8. Remove Duplicates from a Sorted List

Problem: Remove duplicate nodes from a sorted singly linked list so each value appears once.

Requirement: Perform the operation in-place, free duplicate nodes, and use $O(1)$ extra space.

Example: 1 1 2 3 3 3 5 $\rightarrow$ 1 2 3 5

9. Check Whether a List Is a Palindrome

Problem: Determine whether the sequence in a singly linked list reads the same forward and backward.

Requirement: Use $O(1)$ extra space by finding the midpoint and reversing part of the list. Restore the original list before returning.

Example: 1 2 3 2 1 $\rightarrow$ Palindrome

10. Rotate the List to the Right by K

Problem: Rotate a singly linked list to the right by K positions.

Requirement: Use pointer manipulation in $O(n)$ time. Handle K larger than the list length using $K \% n$.

Example: 1 2 3 4 5; $K = 2 \rightarrow$ 4 5 1 2 3

Part B: Doubly Linked List**1. Traverse Forward and Backward**

Problem: Create a doubly linked list and display it from head to tail and tail to head.

Requirement: Maintain head and tail pointers; backward traversal must follow prev links.

Example: 3 6 9 12 -> Forward: 3 6 9 12; Backward: 12 9 6 3

2. Count Even and Odd Values

Problem: Traverse a doubly linked list and count nodes containing even and odd values.

Requirement: Use a single forward traversal and print both counts.

Example: 2 7 8 11 14 -> Even = 3, Odd = 2

3. Insert Before a Target Value

Problem: Insert a node immediately before the first node containing a target value.

Requirement: Handle insertion before the head, in the middle, an empty list, and a missing target. Update next and prev.

Example: 10 30 40; insert 20 before 30 -> 10 20 30 40

4. Delete at a Given Position

Problem: Delete the node at a specified 1-based position.

Requirement: Handle the only node, head, tail, middle, and invalid positions. Update head and tail and free the node.

Example: 4 8 12 16; delete position 3 -> 4 8 16

5. Reverse a Doubly Linked List

Problem: Reverse the list in-place and update both head and tail.

Requirement: Exchange next and prev for every node without allocating replacement nodes. Verify in both directions.

Example: 2 4 6 8 -> Forward: 8 6 4 2; Backward: 2 4 6 8

6. Remove Every Occurrence of a Key

Problem: Delete all nodes containing a given key from a doubly linked list.

Requirement: Correctly reconnect both neighboring nodes and handle matches at either end or throughout the list.

Example: 5 7 5 9 5; key = 5 -> 7 9

7. Insert into a Sorted Doubly Linked List

Problem: Insert a value into an ascending doubly linked list while preserving sorted order.

Requirement: Use the first valid insertion position and correctly update head, tail, next, and prev.

Example: 2 5 8 12; insert 7 -> 2 5 7 8 12

8. Find All Pairs with a Given Sum

Problem: In a sorted list of distinct integers, print every pair whose sum equals a target.

Requirement: Use one pointer at the head and one at the tail. Achieve $O(n)$ time without arrays, sets, or nested loops.

Example: 1 2 4 5 6 8 9; target = 10 $\rightarrow$ (1,9), (2,8), (4,6)

9. Swap Two Nodes Without Swapping Data

Problem: Swap two nodes identified by their values without exchanging the data fields.

Requirement: Relink pointers and handle adjacent nodes, head/tail nodes, identical keys, and missing keys.

Example: 10 20 30 40 50; swap 20 and 50 $\rightarrow$ 10 50 30 40 20

10. Rotate the List Left by K

Problem: Rotate a doubly linked list left by K positions.

Requirement: Use $O(n)$ time and $O(1)$ extra space. Correctly update head, tail, next, and prev; use $K \% n$.

Example: 1 2 3 4 5 6; $K = 2 \rightarrow$ 3 4 5 6 1 2